

COMP7005 Week3 Lab Report

Will Otterbein, A01372608

Summary

This report describes my implementation of the week3 bank server lab.

Obtaining a Program Binary

This section describes how to compile the project.

There are two relevant [makefile](#) recipes, build & clean

```
make build
```

This will create the binary in a ./build directory

```
make clean
```

This will remove the current contents of the ./build directory

You can ignore the run-both recipe I was experimenting w/ stuff.

The output binaries are ./build/bank-server & ./build/bank-client respectively & should be started as follows:

TCP examples

```
./build/bank-server tcp 1234
```

```
# or background
```

```
./build/bank-server tcp 1234 &  
./build/bank-client tcp 1234
```

UDP examples

```
./build/bank-server udp 1234
```

```
# or background
```

```
./build/bank-server udp 1234  
./build/bank-client udp 1234
```

Implementation

This section describes how the programs (client & server) were implemented for use with TCP protocol & UDP protocol.

Client (main)

```
int main(int argc, char** argv)
{
    int pport;

    // initialize application mode all fields to 0
    appmode_t mode = { 0 };

    // determine proto & port from program arguments
    // we need 3 arguments to this program
    // be strict, reject more or less
    if (argc != 3) {
        printf("%s incorrect number of arguments\n", MESSAGE_PREFIX);
        goto error;
    }
    // arg1 protocol type
    if (strcmp(_BANK__TCP_PROTO, argv[1]) == 0) {
        mode.proto = TCP;
    } else if (strcmp(_BANK__UDP_PROTO, argv[1]) == 0) {
        mode.proto = UDP;
    } else {
        printf("%s unknown protocol\n", MESSAGE_PREFIX);
        goto error;
    }
    // arg2 port
    if ((pport = atoi(argv[2])) != 0 && test_port(pport))
        mode.port = pport;
    else {
        printf("%s failed to parse port to integer\n", MESSAGE_PREFIX);
        goto error;
    }

    // sanity check
    print_appmode(&mode);

    // protocol specific socket setup
    switch (mode.proto) {
```

```

        case TCP:
            if (bank_client_tcp(&mode) > 0)
                goto error;
            break;

        case UDP:
            if (bank_client_udp(&mode) > 0)
                goto error;
            break;

        default:
            goto error; // should be an impossibility to g
    }

    return 0;

error:
    printf("error running client program exiting\n");

    return 1;
}

```

```

int main(int argc, char** argv)
{
    int pport;

    // initialize application mode all fields to 0
    appmode_t mode = { 0 };

    // determine proto & port from program arguments
    // we need 3 arguments to this program
    // be strict, reject more or less
    if (argc != 3) {
        printf("%s incorrect number of arguments\n", MESSAGE_PREFIX);
        goto error;
    }
    // arg1 protocol type
    if (strcmp(_BANK__TCP_PROTO, argv[1]) == 0 ) {
        mode.proto = TCP;
    } else if (strcmp(_BANK__UDP_PROTO, argv[1]) == 0) {
        mode.proto = UDP;
    } else {
        printf("%s unknown protocol\n", MESSAGE_PREFIX);
        goto error;
    }
    // arg2 port
    if ((pport = atoi(argv[2])) != 0 && test_port(pport)) {
        mode.port = pport;
    } else {

```

```
        printf("%s failed to parse port to integer\n", MESSAGE_PREFIX);
        goto error;
    }

    // sanity check
    print_appmode(&mode);

    // protocol specific socket setup
    switch (mode.proto) {
        case TCP:
            if (bank_client_tcp(&mode) > 0)
                goto error;
            break;

        case UDP:
            if (bank_client_udp(&mode) > 0)
                goto error;
            break;

        default:
            goto error; // should be an impossibility to get here
    }

    return 0;

error:
    printf("error running client program exiting\n");

    return 1;
}
```

The job of the main method in the client program is to parse the command line arguments into the `mode` instance of `appmode_t` struct & kick start either the TCP or UDP client event loop.

Argument parsing is strict, so the exact count (3) must be met.

Argument 0 is the program name, this is ignored.

Argument 1 is the protocol, this is matched against the `_BANK__TCP_PROTO` and `_BANK__UDP_PROTO` macro strings, matches are recorded the TCP & UDP enumeration values (0, & 1 respectively) & written to `mode.proto`.

Argument 2 is the port of the server, this is parsed from ASCII to integer & the `test_port` helper function of `common.c` is used to determine if the port is within the valid range of ports, defined as follows:

```
#define _BANK__PORT_MIN (1024)
#define _BANK__PORT_MAX (65535)
```

If `mode.proto` is evaluated to be TCP, then `bank_client_tcp` is invoked.

If `mode.proto` is evaluated to be UDP, then `bank_client_udp` is invoked.

Server (main)

```
int main(int argc, char** argv)
{
    int pport;
    appmode_t mode = { 0 };
    bank_t bank = { 0 };

    bank.balance = 1000;

    // parse arguments
    // determine proto & port from program arguments
    // we need 3 arguments to this program
    // be strict, reject more or less

    if (argc != 3) {
        printf("%s incorrect number of arguments\n", MESSAGE_PREFIX);
        goto error;
    }
    // arg1 protocol type
    if (strcmp(_BANK__TCP_PROTO, argv[1]) == 0 ) {
        mode.proto = TCP;
    } else if (strcmp(_BANK__UDP_PROTO, argv[1]) == 0) {
        mode.proto = UDP;
    } else {
        printf("%s unknown protocol\n", MESSAGE_PREFIX);
        goto error;
    }
    // arg2 port
    if ((pport = atoi(argv[2])) != 0 && test_port(pport)) {
        mode.port = pport;
    } else {
        printf("%s failed to parse port to integer\n", MESSAGE_PREFIX);
        goto error;
    }

    print_appmode(&mode);

    // start the bank server according to the proto argument
    printf("starting server w/ proto %d\n", mode.proto);

    switch (mode.proto) {
```

```

        case TCP:
            if (bank_server_tcp(&mode, &bank) != 0)
                goto error;
            break;
        case UDP:
            if (bank_server_udp(&mode, &bank) != 0)
                goto error;
            break;
        default:
            printf("critical server error server attempted start w/
            goto error;
    }

    printf("server finished execution\n");

    return 0;

error:
    printf("error running server program exiting\n");

    return 1;
}

```

```

int main(int argc, char** argv)
{
    int pport;
    appmode_t mode = { 0 };
    bank_t bank = { 0 };

    bank.balance = 1000;

    // parse arguments
    // determine proto & port from program arguments
    // we need 3 arguments to this program
    // be strict, reject more or less

    if (argc != 3) {
        printf("%s incorrect number of arguments\n", MESSAGE_PREFIX);
        goto error;
    }
    // arg1 protocol type
    if (strcmp(_BANK__TCP_PROTO, argv[1]) == 0 ) {
        mode.proto = TCP;
    } else if (strcmp(_BANK__UDP_PROTO, argv[1]) == 0) {
        mode.proto = UDP;
    } else {
        printf("%s unknown protocol\n", MESSAGE_PREFIX);
        goto error;
    }
    // arg2 port
    if ((pport = atoi(argv[2])) != 0 && test_port(pport)) {
        mode.port = pport;
    }
}

```

```

    } else {
        printf("%s failed to parse port to integer\n", MESSAGE_PREFIX);
        goto error;
    }

    print_appmode(&mode);

    // start the bank server according to the proto argument
    printf("starting server w/ proto %d\n", mode.proto);

    switch (mode.proto) {
        case TCP:
            if (bank_server_tcp(&mode, &bank) != 0)
                goto error;
            break;
        case UDP:
            if (bank_server_udp(&mode, &bank) != 0)
                goto error;
            break;
        default:
            printf("critical server error server attempted start w/ non TCP or UDP protocol\n");
            goto error;
    }

    printf("server finished execution\n");

    return 0;

error:
    printf("error running server program exiting\n");

    return 1;
}

```

The job of the main method in the server program is largely the same as the client program, to parse the command line arguments into the mode instance of `appmode_t` struct & kick start either the TCP or UDP server event loop.

Unlike the client program, the main method of the server program also initialized the values of the `bank_t` struct type, the field `balance` is set to 1000 & the flag `should_quit` is zeroed (false).

If `mode.proto` is evaluated to be TCP, then `bank_server_tcp` is invoked.

If `mode.proto` is evaluated to be UDP, then `bank_server_udp` is invoked.

TCP Client

The following sections describe the implementation of the TCP client.

TCP Client setup/teardown

setup

```
int bank_client_tcp(appmode_t* mode)
{
    int sockfd = -1;
    int connectr;
    int pollr;
    int readr;
    int sendr;
    int recvr;

    socklen_t saddr_len;

    struct sockaddr_in saddr = { 0 };
    struct pollfd pfd = { 0 };

    char* rbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));
    char* wbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));

    saddr_len = sizeof(saddr);

    // create TCP socket
    if (getsockfd_tcp(&sockfd) > 0)
        goto error;
    setsockaddr_lb(&saddr, mode->port); // sockaddr -> lb:port

    // connect to server with TCP
    connectr = connect(sockfd, (struct sockaddr*)&saddr, saddr_len);
    if (connectr < 0)
        goto error;

    pfd.fd = STDIN_FILENO; // stdin because we are waiting for
    pfd.events = POLLIN;
    pfd.revents = 0;

    printf("connection to server established entering poll loop.\n")
}
```

teardown & error handling

```
    if (sockfd > 0)
        close(sockfd);

    free(rbuf);
    free(wbuf);

    return BANKCLIENT_OK;

error:
    if (sockfd > 0)
        close(sockfd);

    free(rbuf);
    free(wbuf);

    return BANKCLIENT_ERR;
}
```

```
int bank_client_tcp(appmode_t* mode)
{
    int sockfd = -1;
    int connectr;
    int pollr;
    int readr;
    int sendr;
    int recvr;

    socklen_t saddr_len;

    struct sockaddr_in saddr = { 0 };
    struct pollfd pfd = { 0 };

    char* rbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));
    char* wbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));

    saddr_len = sizeof(saddr);

    // create TCP socket
    if (getsockfd_tcp(&sockfd) > 0)
        goto error;
    setsockaddr_lb(&saddr, mode->port); // sockaddr -> lb:port

    // connect to server with TCP
    connectr = consock(sockfd, (struct sockaddr*)&saddr, saddr_len);
    if (connectr < 0)
        goto error;

    pfd.fd = STDIN_FILENO; // stdin because we are waiting for user input (which is the message)
    pfd.events = POLLIN;
    pfd.revents = 0;

    printf("connection to server established entering poll loop.\n");
```

```

do {
    // event loop ...
} while (1);

if (sockfd > 0)
    close(sockfd);

free(rbuf);
free(wbuf);

return BANKCLIENT_OK;

error:
if (sockfd > 0)
    close(sockfd);

free(rbuf);
free(wbuf);

return BANKCLIENT_ERR;
}

```

First the setup / teardown details before we describe what happens in the event loop.

We declare variables for tracking the returns of syscalls / helpers as well as the `pfd` struct `pollfd` & `saddr` struct `sockaddr_in` (wrapper of struct `sockaddr` for internet, ipv4, address) types. The size of this struct is written to `saddr_len`.

Then the buffers `rbuf` & `wbuf` are allocated, `rbuf` is used to read from STDIN & the responses from the server are written to `wbuf`, they are allocated 256 bytes each as per the `_BANK__BUF_SIZE` macro.

`getsockfd_tcp` wrapper is used to call `socket` w/ the `SOCK_STREAM` parameter w/ interrupt error handling (this is the pattern for most syscall helpers in these programs, which for reference live in `common.c`). Flags are left as default, so the socket is a blocking socket. The sockets file descriptor is written to the `sockfd` variables address.

Then `setsockaddr_lb` is used to set the socket address to be the loopback IP w/ the specified port (which should be the server port). This helper uses `htons` & `htonl` to ensure that the address & port are in the correct byte-ordering for the network stack.

`consock` helper calls `connect` on the now set values of `saddr` & retries if the error returned is `EINTR`. For the client, we use whatever the OS automatically binds as the port.

The last part of the setup is to set the fields of `pfd`, which in the client the `fd` is STDIN & the events we care about are `POLLIN`. `revents` is zeroed.

After the event loop concludes in the normal case, the socket file descriptor is closed, only if it was opened, & the buffers are freed.

The error block also closes the socket, only if it was opened (ie it no longer has the initialized value of -1). `rbuf` & `wbuf` are freed as well.

TCP Client event loop

```
do {
    pollr = poll(&pfd, 1, -1); // poll on stdin (messages)

    if (pollr > 0) {
        if (pfd.revents & ( POLLNVAL | POLLERR )) {
            goto error;

        } else if (pfd.revents & ( POLLIN )) {
            memset(rbuf, 0, _BANK__BUF_SIZE);
            memset(wbuf, 0, _BANK__BUF_SIZE);

            // read command from stdin
            readr = readstdin(pfd.fd, rbuf, (size_t)_BANK__BUF_SIZE);
            if (readr < 0) {
                break;
            }
            if (readr == 0) {
                break;
            }
            // TCP send command the the server
            sendr = send_all(sockfd, rbuf, (ssize_t)readr);
            if (sendr < 0) {
                break;
            }
            // wait & then receive the reply from the server
            recvr = recv_all(sockfd, wbuf, (ssize_t)_BANK__BUF_SIZE);
            if (recvr < 0) {
                break;
            }
            if (strchr(wbuf, '\n') == NULL) {
                printf("invalid reply\n");
                continue;
            }

            printf("%s\n", wbuf); // reply from the server
            // if the server replied with the shutdown message
            // break out of the event loop
            if (strncmp(_BANK__SERVER_SHUTDOWN_MSG_PREFIX, wbuf, str
                break;
            }
        }
    }
}
```

```

        continue;
    }
} else if (pollr == 0) {
    continue;
} else {
    if (errno == EINTR)
        continue;
    printf("error with poll\n");
    goto error;
}
} while (1);

```

```

int bank_client_tcp(appmode_t* mode)
{
    // setup ...
    do {
        pollr = poll(&pfd, 1, -1); // poll on stdin (messages)

        if (pollr > 0) {
            if (pfd.revents & ( POLLNVAL | POLLERR )) {
                goto error;
            }

            } else if (pfd.revents & ( POLLIN )) {
                memset(rbuf, 0, _BANK__BUF_SIZE);
                memset(wbuf, 0, _BANK__BUF_SIZE);

                // read command from stdin
                readr = readstdin(pfd.fd, rbuf, (size_t)_BANK__BUF_SIZE);
                if (readr < 0) {
                    break;
                }
                if (readr == 0) {
                    break;
                }
                // TCP send command the the server
                sendr = send_all(sockfd, rbuf, (ssize_t)readr);
                if (sendr < 0) {
                    break;
                }
                // wait & then receive the reply from the server
                recvr = recv_all(sockfd, wbuf, (ssize_t)_BANK__BUF_SIZE);
                if (recvr < 0) {
                    break;
                }
                if (strchr(wbuf, '\n') == NULL) {
                    printf("invalid reply\n");
                    continue;
                }

                printf("%s\n", wbuf); // reply from the server
                // if the server replied with the shutdown message
                // break out of the event loop
                if (strncmp(_BANK__SERVER_SHUTDOWN_MSG_PREFIX, wbuf, strlen(_BANK__SERVER_SHUTDOWN_MSG_PR
                    break;

                continue;
            }
        } else if (pollr == 0) {
            continue;
        } else {

```

```
        if (errno == EINTR)
            continue;
        printf("error with poll\n");
        goto error;
    }
} while (1);
// teardown ...
}
```

Assuming that the setup completed successfully, the TCP client event loop begins executing.

The event loop implementation is based on polling STDIN, since that is where it is expected that commands will be written to.

First, handling the returns of the `poll` syscall itself, if its > 0 we need to process the input, however if it's zero then the timeout was reached (not relevant in this case w/ `poll` being called w/ a timeout of -1), & if there was an error w/ `poll` (excluding `EINTR`), we jump to error processing.

If there is input to process, then the buffers (`rbuf` & `wbuf`) are reset & the input is read into `rbuf` using the `readstdin` helper, which is responsible for re-attempting the read syscall if the error returned is `EINTR` & injecting the null terminator after the last read byte (ensuring that the input is a correct string).

The `send_all` helper is invoked, this helper does more than just re-invoke the syscall if the error is `EINTR`, it continues to write until all bytes of the message have been sent (since TCP is not guaranteed to do this with one send call), which is when the amount sent equals the length of the full message. If sending fails, this means an underlying connection error & so the client is terminated.

`recv_all` will put the client into a blocked state until the server reply message becomes available to read, & `recv_all` similarly wraps the `recv` syscall reading until the buffer limit 256 or until the '\n' end-of-message sentinel is encountered (again, necessary because TCP might not have the entire message ready with one `recv` call) & returning the total number of bytes in the received message.

This message is logged & if it is determined to be the BYE reply of the server, then the client begins termination.

TCP Server

The following sections describe the implementation of the TCP server.

TCP Server setup/teardown

setup

```
/**
bank server implementation with sock_stream (TCP) underlying protocol
*/
int bank_server_tcp(appmode_t* am, bank_t* bank)
{
    int sockfd = -1;
    int consockfd = -1;

    int listenr;
    int acceptr;
    int pollr;
    int sendr;
    int readr;

    struct sockaddr_in saddr      = { 0 };
    struct pollfd pfd             = { 0 };

    char* rbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));
    char* wbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));

    // tcp socket setup
    if (getsockfd_tcp(&sockfd) > 0)
        goto error;
    setsockaddr_lb(&saddr, am->port); // sockaddr -> lb:port
    if (bindsock(sockfd, (struct sockaddr*)&saddr, sizeof(saddr)))
        goto error;

    // tcp listen for connections w/ queue size 1
    listenr = listen(sockfd, 1);
    if (listenr != 0)
        goto error;
```

teardown & error handling

```

    if (sockfd > 0)
        close(sockfd);

    free(rbuf);
    free(wbuf);

    return BANKCLIENT_OK;

error:
    if (sockfd > 0)
        close(sockfd);

    free(rbuf);
    free(wbuf);

    return BANKCLIENT_ERR;
}

```

```

/**
bank server implementation with sock_stream (TCP) underlying protocol.
*/
int bank_server_tcp(appmode_t* am, bank_t* bank)
{
    int sockfd = -1;
    int consockfd = -1;

    int listenr;
    int acceptr;
    int pollr;
    int sendr;
    int readr;

    struct sockaddr_in saddr = { 0 };
    struct pollfd pfd = { 0 };

    char* rbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));
    char* wbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));

    // tcp socket setup
    if (getsockfd_tcp(&sockfd) > 0)
        goto error;
    setsockaddr_lb(&saddr, am->port); // sockaddr -> lb:port
    if (bindsock(sockfd, (struct sockaddr*)&saddr, sizeof(saddr)))
        goto error;

    // tcp listen for connections w/ queue size 1
    listenr = listen(sockfd, 1);
    if (listenr != 0)
        goto error;

    // tcp server loop
    do {

```

```
        // event loop ...
    } while (1);

    printf("server program terminating\n");

    if (sockfd >= 0)
        close(sockfd);
    if (consockfd >= 0)
        close(consockfd);
    free(rbuf);
    free(wbuf);

    return BANKSERV_OK;

error:
    printf("there was an error running the server quitting server event loop\n");

    if (sockfd >= 0)
        close(sockfd);
    if (consockfd >= 0)
        close(consockfd);
    free(rbuf);
    free(wbuf);

    return BANKSERV_ERR;
}
```

Similar to the TCP client setup, but this time 2 file descriptors are declared, `sockfd` & `consockfd`. Unlike the client, the server binds the loopback address & the port specified in `am appmode_t` pointer (which if you recall was initialized from the arguments) to `sockfd` via the `bindsock` helper, which takes care of `EINTR`. `rbuf` & `wbuf` names are reused for buffers that serve similar purposes, except this time contents are read into `rbuf` from the eventually client connected socket represented by `consockfd`.

The last part of setting the TCP server loop up involves setting the state of the `sockfd` to listen, this we do directly & jump to error processing immediately if this fails.

Error processing involves checking that `sockfd` & `consockfd` were initialized, if they were then they are closed. The `rbuf` & `wbuf` are freed.

TCP Server event loop

```
// tcp server loop
do {
    // tcp connection acceptance (1 client at a time)
    consockfd = acceptcon(sockfd);
    if (consockfd < 0)
        break;

    // tcp connection poll loop
    pfd.fd      = consockfd;
    pfd.events  = POLLIN;
    pfd.revents = 0;

    do {
        pollr = poll(&pfd, 1, -1);

        // handle poll stuff
        if (pollr > 0) {
            if (pfd.revents & ( POLLNVAL | POLLERR )) {
                goto error;
            } else if (pfd.revents & ( POLLHUP )) {
                break;
            } else {
                memset(rbuf, 0, _BANK__BUF_SIZE);
                memset(wbuf, 0, _BANK__BUF_SIZE);

                // read message
                readr = recv_all(pfd.fd, rbuf, (ssize_t)_BANK__BUF_SIZE);
                if (readr < 0) {
                    printf("error reading from client socket closing connection\n");
                    break;
                } else if (readr == 0) {
                    printf("server read EOF from socket connection connection\n");
                    break;
                }
                // check if the command is valid
                // terminate the connection if the command is not valid
                if (strchr(rbuf, '\n') == NULL) {
                    printf("invalid command\n");
                    break;
                }
                // app logic (common)
                bank_processing(bank, rbuf, wbuf);
                // send the reply
                sendr = send_all(pfd.fd, wbuf, (ssize_t)strlen(wbuf));
                if (sendr < 0) {
```

```

        printf("error writing to client socket closing conn
        break;
    }
    // time to end the server
    if (bank->should_quit) {
        printf("QUIT command received closing connection &
        break;
    }

    continue;
}
} else if (pollr == 0) {
    continue;
} else {
    if (errno == EINTR)
        continue;
    printf("error with poll\n");
    goto error;
}
} while (1);

close(consockfd);

// time to end the server
if (bank->should_quit)
    break;

} while (1);

```

/**
bank server implementation with sock_stream (TCP) underlying protocol.

*/

int bank_server_tcp(appmode_t* am, bank_t* bank)

{

// setup ...

// tcp server loop

do {

// tcp connection acceptance (1 client at a time)

consockfd = acceptcon(sockfd);

if (consockfd < 0)

break;

// tcp connection poll loop

pfd.fd = consockfd;

pfd.events = POLLIN;

pfd.revents = 0;

do {

pollr = poll(&pfd, 1, -1);

// handle poll stuff

if (pollr > 0) {

if (pfd.revents & (POLLNVAL | POLLERR)) {

goto error;

```

    } else if (pfd.revents & ( POLLHUP )) {
        break;
    } else {
        memset(rbuf, 0, _BANK__BUF_SIZE);
        memset(wbuf, 0, _BANK__BUF_SIZE);

        // read message
        readr = recv_all(pfd.fd, rbuf, (ssize_t)_BANK__BUF_SIZE);
        if (readr < 0) {
            printf("error reading from client socket closing connection\n");
            break;
        } else if (readr == 0) {
            printf("server read EOF from socket connection connection closed\n");
            break;
        }
        // check if the command is valid
        // terminate the connection if the command is not valid
        if (strchr(rbuf, '\n') == NULL) {
            printf("invalid command\n");
            break;
        }
        // app logic (common)
        bank_processing(bank, rbuf, wbuf);
        // send the reply
        sendr = send_all(pfd.fd, wbuf, (ssize_t)strlen(wbuf));
        if (sendr < 0) {
            printf("error writing to client socket closing connection\n");
            break;
        }
        // time to end the server
        if (bank->should_quit) {
            printf("QUIT command received closing connection & terminating server\n");
            break;
        }

        continue;
    }
} else if (pollr == 0) {
    continue;
} else {
    if (errno == EINTR)
        continue;
    printf("error with poll\n");
    goto error;
}
} while (1);

close(consockfd);

// time to end the server
if (bank->should_quit)
    break;

} while (1);

// error/teardown ...
}

```

The TCP server involves two layers of looping, one on the acceptcon accept wrapper, safely dealing with EINTR, once a connection is established the the struct pollfd pfd is setup with the file descriptor of the connected client socket (consockfd).

The inner layer of looping involves polling for input with this now connected client socket. Once poll stops blocking & its return reports that there is data to be read, we read this using the same recv_all syscall wrapper,

ensuring that all bytes available are read. If the message does not contain a newline, it is considered as invalid & we terminate the connection by breaking this inner loop, returning to the outer loop to wait for a new client to connect.

If the message is valid, then we pass it off to `bank_processing`, which will handle the application logic & write a reply to `wbuf`. Now the TCP server uses the `send_all` helper to write this message to the client.

The `bank_t` struct, which if you recall from the `main` info, defines the applications state, has a flag `should_quit` which may have been set to true via `bank_processing`, if this is the case, then the inner connection loop is broken (which results if the `consockfd` being closed), & the outer accept loop is also broken leading to the TCP server teardown.

UDP Client

The following sections describe the implementation of the UDP client.

UDP Client setup/teardown

setup

```
int bank_client_udp(appmode_t* mode)
{
    int sockfd = -1;
    int connectr;
    int pollr;
    int readr;
    int sendr;
    int recvr;

    socklen_t saddr_len;
    socklen_t paddr_len;

    struct sockaddr_in saddr = { 0 };
    struct sockaddr_in paddr = { 0 }; // write reply address to this instead
    struct pollfd pfd = { 0 };

    char* rbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));
    char* wbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));

    saddr_len = sizeof(saddr);
    paddr_len = sizeof(paddr);

    // create UDP socket
    if (getsockfd_udp(&sockfd) > 0)
        goto error;
    setsockaddr_lb(&saddr, mode->port); // sockaddr → lb:port

    pfd.fd = STDIN_FILENO; // stdin because we are waiting for user input
    pfd.events = POLLIN;
    pfd.revents = 0;

    printf("entering poll loop.\n");
```

teardown & error handling

```
    printf("client program terminating\n");

    if (sockfd > 0)
        close(sockfd);

    free(rbuf);
    free(wbuf);

    return BANKCLIENT_OK;

error:
    if (sockfd > 0)
        close(sockfd);

    free(rbuf);
    free(wbuf);

    return BANKCLIENT_ERR;
}
```

```
int bank_client_udp(appmode_t* mode)
{
    int sockfd = -1;
    int connectr;
    int pollr;
    int readr;
    int sendr;
    int recvr;

    socklen_t saddr_len;
    socklen_t paddr_len;

    struct sockaddr_in saddr = { 0 };
    struct sockaddr_in paddr = { 0 }; // write reply address to this instead of the servers known address
    struct pollfd pfd = { 0 };

    char* rbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));
    char* wbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));

    saddr_len = sizeof(saddr);
    paddr_len = sizeof(paddr);

    // create UDP socket
    if (getsockfd_udp(&sockfd) > 0)
        goto error;
    setsockaddr_lb(&saddr, mode->port); // sockaddr -> lb:port

    pfd.fd      = STDIN_FILENO; // stdin because we are waiting for user input (which is the message)
    pfd.events  = POLLIN;
    pfd.revents = 0;

    printf("entering poll loop.\n");

    do {
        // event loop ...
    } while (1);
}
```

```
    printf("client program terminating\n");

    if (sockfd > 0)
        close(sockfd);

    free(rbuf);
    free(wbuf);

    return BANKCLIENT_OK;

error:
    if (sockfd > 0)
        close(sockfd);

    free(rbuf);
    free(wbuf);

    return BANKCLIENT_ERR;
}
```

Similar to the TCP client, except there is no need to invoke connect on the socket & the socket is created using the getsockfd_udp helper setting SOCK_DGRAM as the type.

Poll is again setup w/ STDIN as this is still where we are expecting the messages to come from.

Regarding error handling & teardown, if sockfd was opened, then it is closed & the buffers rbuf & wbuf are freed.

UDP Client event loop

```
do {
    pollr = poll(&pfd, 1, -1); // poll on stdin (messages)

    if (pollr > 0) {
        if (pfd.revents & ( POLLNVAL | POLLERR )) {
            goto error;

        } else if (pfd.revents & ( POLLIN )) {
            memset(rbuf, 0, _BANK__BUF_SIZE);
            memset(wbuf, 0, _BANK__BUF_SIZE);

            // read command from stdin
            readr = readstdin(pfd.fid, rbuf, _BANK__BUF_SIZE);
            if (readr < 0) {
                break;
            }
            if (readr == 0) {
                break;
            }
            // UDP send command the the server
            sendr = sendto_all(sockfd, rbuf, readr,
                               (struct sockaddr*)&saddr, saddrlen);
            if (sendr < 0) {
                break;
            }
            // wait & then receive the reply from the server
            recvr = recvfrom_all(sockfd, wbuf, _BANK__BUF_SIZE,
                                 (struct sockaddr*)&paddr, &paddr_len);
            if (recvr < 0) {
                break;
            }
            if (recvr == 0) {
                printf("server closed the connection\n");
                break;
            }
            if (strchr(wbuf, '\n') == NULL) {
                printf("invalid reply\n");
                continue;
            }

            printf("%s\n", wbuf); // reply from the server
            // if the server replied with the shutdown message
            // break out of the event loop
            if (strncmp(_BANK__SERVER_SHUTDOWN_MSG_PREFIX, wbuf, strlen(
                break;
```

```

        continue;
    }
} else if (pollr == 0) {
    continue;
} else {
    if (errno == EINTR)
        continue;
    printf("error with poll\n");
    goto error;
}
} while (1);

```

```

int bank_client_udp(appmode_t* mode)
{
    // setup ...
    do {
        pollr = poll(&pfd, 1, -1); // poll on stdin (messages)

        if (pollr > 0) {
            if (pfd.revents & ( POLLNVAL | POLLERR )) {
                goto error;

            } else if (pfd.revents & ( POLLIN )) {
                memset(rbuf, 0, _BANK__BUF_SIZE);
                memset(wbuf, 0, _BANK__BUF_SIZE);

                // read command from stdin
                readr = readstdin(pfd.fd, rbuf, _BANK__BUF_SIZE);
                if (readr < 0) {
                    break;
                }
                if (readr == 0) {
                    break;
                }
                // UDP send command the the server
                sendr = sendto_all(sockfd, rbuf, readr,
                                   (struct sockaddr*)&saddr, saddr_len);
                if (sendr < 0) {
                    break;
                }
                // wait & then receive the reply from the server
                recvr = recvfrom_all(sockfd, wbuf, _BANK__BUF_SIZE,
                                     (struct sockaddr*)&paddr, &paddr_len);
                if (recvr < 0) {
                    break;
                }
                if (recvr == 0) {
                    printf("server closed the connection\n");
                    break;
                }
                if (strchr(wbuf, '\n') == NULL) {
                    printf("invalid reply\n");
                    continue;
                }

                printf("%s\n", wbuf); // reply from the server
                // if the server replied with the shutdown message
                // break out of the event loop
                if (strncmp(_BANK__SERVER_SHUTDOWN_MSG_PREFIX, wbuf, strlen(_BANK__SERVER_SHUTDOWN_MSG_PR

```

```

        break;

        continue;
    }
} else if (pollr == 0) {
    continue;
} else {
    if (errno == EINTR)
        continue;
    printf("error with poll\n");
    goto error;
}
} while (1);
// error/teardown ...
}

```

`poll` is invoked with an infinite timeout, once `poll` returns, if the return indicates that there's data to be read, when we process it. Otherwise if 0 is returned by `poll`, we go to the next iteration of the client processing loop & if an error is returned, we confirm its not from `EINTR`.

STDIN is read into the `rbuf` via `readstdin` helper, but the message is then sent to the server using the `sendto_all` wrapper, which means the destination address `saddr` & its length must be passed as well. `sendto_all` handles writing the message to the server over the socket, the wrapper checks if there's an error that the error is `EINTR` & re-attempts, otherwise the message is assumed to be sent in one go since this is UDP.

After the message is successfully sent, the client begins waiting for a reply via `recvfrom_all` which does similar processing as `sendto_all` in that the message is retransmitted if the syscall results in an `EINTR` error, otherwise successful returns assume the entire message is sent (since this is UDP).

The server's reply, written to `wbuf`, is printed & checked to see if its the termination message (which if it is, the client event loop is broken & teardown begins).

UDP Server

The following sections describe the implementation of the UDP server.

UDP Server setup/teardown

setup

```
/**
bank server implementation with sock_dgram (UDP) underlying protocol.
*/
int bank_server_udp(appmode_t* am, bank_t* bank)
{
    int sockfd = -1;

    int listenr;
    int acceptr;
    int pollr;
    int sendr;
    int readr;

    struct sockaddr_in saddr    = { 0 };
    struct sockaddr_in caddr    = { 0 };           // peers address
    struct pollfd pfd          = { 0 };

    socklen_t caddr_len = sizeof(caddr);

    char* rbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));
    char* wbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));

    // udp socket setup
    if (getsockfd_udp(&sockfd) > 0)
        goto error;
    setsockaddr_lb(&saddr, am->port); // sockaddr → lb:port
    if (bindsock(sockfd, (struct sockaddr*)&saddr, sizeof(saddr)))
        goto error;

    // socket poll loop
    pfd.fd      = sockfd;
    pfd.events  = POLLIN;
    pfd.revents = 0;
```

teardown & error handling

```

    printf("server program terminating\n");

    if (sockfd ≥ 0)
        close(sockfd);
    free(rbuf);
    free(wbuf);

    return BANKSERV_OK;
error:
    printf("there was an error running the server quitting server event loop\n");

    if (sockfd ≥ 0)
        close(sockfd);
    free(rbuf);
    free(wbuf);

    return BANKSERV_ERR;
}

```

```

/**
bank server implementation with sock_dgram (UDP) underlying protocol.
*/
int bank_server_udp(appmode_t* am, bank_t* bank)
{
    int sockfd = -1;

    int listenr;
    int acceptr;
    int pollr;
    int sendr;
    int readr;

    struct sockaddr_in saddr    = { 0 };
    struct sockaddr_in caddr    = { 0 };           // peers address
    struct pollfd pfd          = { 0 };

    socklen_t caddr_len = sizeof(caddr);

    char* rbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));
    char* wbuf = (char*)calloc(_BANK__BUF_SIZE, sizeof(char));

    // udp socket setup
    if (getsockfd_udp(&sockfd) > 0)
        goto error;
    setsockaddr_lb(&saddr, am->port); // sockaddr -> lb:port
    if (bindsock(sockfd, (struct sockaddr*)&saddr, sizeof(saddr)))
        goto error;

    // socket poll loop
    pfd.fd      = sockfd;
    pfd.events  = POLLIN;
    pfd.revents = 0;

    do {
        // event loop ...
    } while (1);
}

```

```
    } while (1);

    printf("server program terminating\n");

    if (sockfd >= 0)
        close(sockfd);
    free(rbuf);
    free(wbuf);

    return BANKSERV_OK;

error:
    printf("there was an error running the server quitting server event loop\n");

    if (sockfd >= 0)
        close(sockfd);
    free(rbuf);
    free(wbuf);

    return BANKSERV_ERR;
}
```

UDP server setup unlike the TCP server setup involves no connection. The socket is acquired via `getsockfd_udp` & bound via `bindsock` to the loopback address & configured port in much the same way.

`struct sockaddr_in caddr` is declared alongside the `socklen_t caddr_len` which is used when reading client messages as the variable which stores this particular clients address in order that replies be sent to them later.

`rbuf` & `wbuf` are allocated again, for reading the incoming messages into & writing processed response messages to.

`pf` is setup w/ the bound socket & the event loop begins.

If an error is detected during setup, the error processing involves closing the `sockfd` if it was opened & freeing the `rbuf` & `wbuf` buffers.

UDP Server event loop

```
do {
    pollr = poll(&pfd, 1, -1);

    // handle poll stuff
    if (pollr > 0) {
        if (pfd.revents & ( POLLNVAL | POLLERR )) {           // POLLHUP r
            goto error;
        } else {
            memset(rbuf, 0, _BANK__BUF_SIZE);
            memset(wbuf, 0, _BANK__BUF_SIZE);

            // read message
            readr = recvfrom_all(pfd.fd, rbuf, _BANK__BUF_SIZE,
                                (struct sockaddr*)&caddr, &caddr_len);
            if (readr < 0) {
                printf("there was an error reading the clients message\n");
                continue;
            }
            if (strchr(rbuf, '\n') == NULL) {
                printf("invalid command\n");
                continue;
            }

            // app logic (common)
            bank_processing(bank, rbuf, wbuf);

            // send the reply
            sendr = sendto_all(pfd.fd, wbuf, strlen(wbuf),
                              (struct sockaddr*)&caddr, caddr_len);
            if (sendr < 0) {
                printf("there was an error sending reply message to the");
                continue;
            }

            // time to end the server
            if (bank->should_quit) {
                printf("quit command recieved terminating server\n");
                break;
            }

            continue;
        }
    } else if (pollr == 0) {
        continue;
    }
}
```

```

    } else {
        if (errno == EINTR)
            continue;
        printf("error with poll\n");
        goto error;
    }
} while (1);

```

```

/**
bank server implementation with sock_dgram (UDP) underlying protocol.
*/
int bank_server_udp(appmode_t* am, bank_t* bank)
{
    // setup ...
    do {
        pollr = poll(&pfd, 1, -1);

        // handle poll stuff
        if (pollr > 0) {
            if (pfd.revents & ( POLLNVAL | POLLERR )) {          // POLLHUP not raised (UDP therefore no c
                goto error;
            }

            } else {
                memset(rbuf, 0, _BANK__BUF_SIZE);
                memset(wbuf, 0, _BANK__BUF_SIZE);

                // read message
                readr = recvfrom_all(pfd.fd, rbuf, _BANK__BUF_SIZE,
                    (struct sockaddr*)&caddr, &caddr_len);
                if (readr < 0) {
                    printf("there was an error reading the clients message\n");
                    continue;
                }
                if (strchr(rbuf, '\n') == NULL) {
                    printf("invalid command\n");
                    continue;
                }

                // app logic (common)
                bank_processing(bank, rbuf, wbuf);

                // send the reply
                sendr = sendto_all(pfd.fd, wbuf, strlen(wbuf),
                    (struct sockaddr*)&caddr, caddr_len);
                if (sendr < 0) {
                    printf("there was an error sending reply message to the client\n");
                    continue;
                }

                // time to end the server
                if (bank->should_quit) {
                    printf("quit command recieved terminating server\n");
                    break;
                }

                continue;
            }
        } else if (pollr == 0) {
            continue;
        } else {
            if (errno == EINTR)
                continue;
            printf("error with poll\n");

```

```
        goto error;
    }
} while (1);
// error/teardown ...
}
```

If the setup is successful, the UDP server event loop begins. Recall that `sockfd` was set as the target file descriptor to poll against. If `poll` returns some `n > 0` then there is a message to read from the client. Otherwise, if `poll` returns 0 (timeout) we continue awaiting incoming data & if there is an error with `poll`, first we check that its not the interrupt error & if its not then we jump to error processing.

The message from the client is recieved via `recvfrom_all`, with the clients address being written to `caddr`. Upon successful reading of the message, its passed to `bank_processing` & the reply is written to `wbuf`. This reply is sent to the address saved in `caddr` via `sendto_all`.

If `bank_processing` determines that the message from the client was the 'QUIT' message, this is indicated in the `bank` struct `should_quit` field being set, if this is the case then the final reply is sent ('BYE' reply), the UDP server event loop is broken & the teardown procedure begins.

TCP/UDP Server message processing

Once the `rbuf` is populated, either protocol must process messages in the same way & write replies in the same way to `wbuf`, so this is abstracted to the `bank_processing` function.

```

/**
Takes message from the client & processes the request
& writes a response message.
    bank_t* bank pointer to bank struct (bank state)
    char* request
    char* response
*/
void bank_processing(bank_t* bank, const char* request, char* response) {
    int t = -1;

    if (strncmp(_BANK__BALANCE_MSG_PREFIX,
        request, strlen(_BANK__BALANCE_MSG_PREFIX)) == 0) {

        sprintf(response, _BANK__REPLY_MSG_PREFIX " " _BANK__BALANCE_MSG_PREFIX
            bank->balance);

    } else if (strncmp(_BANK__DEPOSIT_MSG_PREFIX,
        request, strlen(_BANK__DEPOSIT_MSG_PREFIX)) == 0) {

        if (sscanf(request, _BANK__DEPOSIT_MSG_PREFIX " %d", &t) == 1) {
            bank->balance += t;
            sprintf(response, _BANK__REPLY_MSG_PREFIX " " _BANK__BALANCE_MSG_PREFIX
                bank->balance);
        } else {
            sprintf(response, "bad command\n");
        }

    } else if (strncmp(_BANK__WITHDRAW_MSG_PREFIX,
        request, strlen(_BANK__WITHDRAW_MSG_PREFIX)) == 0) {

        if (sscanf(request, _BANK__WITHDRAW_MSG_PREFIX " %d", &t) == 1) {
            if (t <= bank->balance) {
                bank->balance -= t;
                sprintf(response, _BANK__REPLY_MSG_PREFIX " " _BANK__BALANCE_MSG_PREFIX
                    bank->balance);
            } else {
                sprintf(response, "ERR INSUFFICIENT_FUNDS\n");
            }
        } else {
            sprintf(response, "bad command\n");
        }

    } else if (strncmp(_BANK__QUIT_MSG,
        request, strlen(_BANK__QUIT_MSG)) == 0) {

        sprintf(response, _BANK__SERVER_SHUTDOWN_MSG_PREFIX "\n");
    }
}

```

```

        bank->should_quit = 1;

    } else {
        sprintf(response, "unknown message type\n");
    }
}

/**
 * Takes message from the client & processes the request
 * & writes a response message.
 *   bank_t* bank pointer to bank struct (bank state)
 *   char* request
 *   char* response
 */
void bank_processing(bank_t* bank, const char* request, char* response) {
    int t = -1;

    if (strncmp(_BANK__BALANCE_MSG_PREFIX,
        request, strlen(_BANK__BALANCE_MSG_PREFIX)) == 0) {

        sprintf(response, _BANK__REPLY_MSG_PREFIX " " _BANK__BALANCE_MSG_PREFIX " %d\n",
            bank->balance);

    } else if (strncmp(_BANK__DEPOSIT_MSG_PREFIX,
        request, strlen(_BANK__DEPOSIT_MSG_PREFIX)) == 0) {

        if (sscanf(request, _BANK__DEPOSIT_MSG_PREFIX " %d", &t) == 1) {
            bank->balance += t;
            sprintf(response, _BANK__REPLY_MSG_PREFIX " " _BANK__BALANCE_MSG_PREFIX " %d\n",
                bank->balance);
        } else {
            sprintf(response, "bad command\n");
        }

    } else if (strncmp(_BANK__WITHDRAW_MSG_PREFIX,
        request, strlen(_BANK__WITHDRAW_MSG_PREFIX)) == 0) {

        if (sscanf(request, _BANK__WITHDRAW_MSG_PREFIX " %d", &t) == 1) {
            if (t <= bank->balance) {
                bank->balance -= t;
                sprintf(response, _BANK__REPLY_MSG_PREFIX " " _BANK__BALANCE_MSG_PREFIX " %d\n",
                    bank->balance);
            } else {
                sprintf(response, "ERR INSUFFICIENT_FUNDS\n");
            }
        } else {
            sprintf(response, "bad command\n");
        }

    } else if (strncmp(_BANK__QUIT_MSG,
        request, strlen(_BANK__QUIT_MSG)) == 0) {

        sprintf(response, _BANK__SERVER_SHUTDOWN_MSG_PREFIX "\n");
        bank->should_quit = 1;

    } else {
        sprintf(response, "unknown message type\n");
    }
}

```

Messages are processed from `rbuf`, in this method `request`, the lexicon is defined using macros & prefix matching is used to determine which message has been passed.

For the messages that accept some number input, `sscanf` is used to extract this value to the `int t`, if this is successful then the corresponding operation (deposit -> addition, withdraw -> subtraction) is performed against the `bank_t` bank struct, whose address is an input to the function. In the case of withdrawing, firstly it is determined whether the amount being withdrawn exceeds that which is in the bank. If so, then an error message is written to `wbuf`. Writing messages to `wbuf` is done via `sprintf`, which can similarly to `sscanf` use format strings with the current value of the bank balance.

Test Results

Last but certainly not least, here are the test results w/ the TCP & UDP versions of the program.

Results of TCP tests:

```
otter@W0surface:~/src/comp7005/labs/week3_a01372608
$ ./build/bank-server tcp 1234
ADDR:0,PROTO:0,PORT:1234
starting server w/ proto 0
QUIT command received closing connection & terminating server
server program terminating
server finished execution
otter@W0surface:~/src/comp7005/labs/week3_a01372608
$ █

otter@W0surface:~/src/comp7005/
$ ./build/bank-client tcp 1234
ADDR:0,PROTO:0,PORT:1234
connection to server establishe
DEPOSIT 100
OK BALANCE 1100

WITHDRAW 100
OK BALANCE 1000

BALANCE
OK BALANCE 1000

WITHDRAW 10000
ERR INSUFFICIENT_FUNDS

BALANCE
OK BALANCE 1000

QUIT
BYE

otter@W0surface:~/src/comp7005/
$ █
```

Results of UDP tests:

```
otter@W0surface:~/src/comp7005/labs/week3_a01372608
$ ./build/bank-server udp 1234
ADDR:0,PROTO:1,PORT:1234
starting server w/ proto 1
quit command recieved terminating server
server program terminating
server finished execution
otter@W0surface:~/src/comp7005/labs/week3_a01372608
$ █
```

```
otter@W0surface:~/src/comp7005/
$ ./build/bank-client udp 1234
ADDR:0,PROTO:1,PORT:1234
entering poll loop.
DEPOSIT 100
OK BALANCE 1100

WITHDRAW 100
OK BALANCE 1000

BALANCE
OK BALANCE 1000

WITHDRAW 10000
ERR INSUFFICIENT_FUNDS

BALANCE
OK BALANCE 1000

QUIT
BYE

client program terminating
otter@W0surface:~/src/comp7005/
```